

</>

Master in fullstack development e AI

di Lorenzo Sottile



News Hub

High-Performance Angular Application
with RxJS & Standalone Components

[Live Demo](#)

[Repository GitHub](#)

La Sfida Tecnica

API Frammentata

L'API ufficiale di Hacker News non fornisce un end-point aggregato, imponendo una struttura a due step:

- Index Fetch: GET /newstories.json restituisce un array di ~500 ID grezzi (es. [123, 124, 125...]).
- Item Details: GET /item/{id}.json richiede una chiamata HTTP singola per ogni notizia.

Obiettivo

Evitare colli di bottiglia e garantire un'esperienza utente fluida senza scaricare 500 dettagli sequenzialmente all'avvio.

"N+1 Problem" & Waterfall Effect

Un approccio ingenuo (iterare su tutti i 500 ID) causerebbe problemi:

- Network Congestion: Il browser limiterebbe le centinaia di richieste simultanee, creando una coda (queue) lunghissima.
- UI Freeze: Il rendering della pagina verrebbe bloccato fino al completamento di tutte le chiamate.
- Spreco di Dati: Scaricare 500 notizie quando l'utente ne legge mediamente solo 10-20 è inefficiente (Over-fetching).

La Soluzione (Reactive Data Fetching)

Orchestrazione Reattiva con RxJS

Parallelismo e Validazione Dati

- Service Integration: L'app delega le chiamate a HackerNewsService, mantenendo il componente pulito.
- Parallel Execution: Utilizzo di forkJoin per eseguire le 10 chiamate del batch in parallelo.
- Data Integrity: Una volta ricevuti i dati, viene applicato un filtro rigoroso per escludere notizie nulle, cancellate (deleted) o morte (dead), garantendo che l'utente veda solo contenuti validi.

```
// Creazione richieste parallele con gestione errori granulare
const requests = idsToLoad.map(id =>
  this.newsService.getNewsItem(id).pipe(
    timeout(3000),           // Timeout di sicurezza
    catchError(_ => of(null)) // Evita il crash dell'intero batch
  )
);
```

```
// Esecuzione e Filtro
forkJoin(requests).subscribe(stories => {
  const validStories = stories.filter(s =>
    s !== null && !!s.title && !s.deleted && !s.dead
  );
  this.displayedStories = [...this.displayedStories,
    ...validStories];
});
```

Architecture & Patterns

Architettura Moderna con Angular Standalone

- **Standalone Components:** L'applicazione abbandona gli NgModules per una struttura più snella e modulare (`standalone: true`), riducendo la complessità di configurazione.
- **Functional Dependency Injection:** Utilizzo della funzione `inject()` invece del costruttore classico. Questo rende il codice più leggibile e safe, specialmente per l'iniezione di `HackerNewsService`.
- **Manual Change Detection:**
 1. L'uso esplicito di `this.cd.detectChanges()` all'interno delle subscription asincrone garantisce che la UI si aggiorni esattamente quando i dati sono pronti.
 2. Questo approccio previene cicli di rendering inutili e assicura che il DOM rifletta lo stato dei dati anche durante operazioni asincrone (`forkJoin`).

User Experience & State Management

UX Resiliente e Feedback Utente

Strategia di UI ("Don't Make Me Think"):

- Dual Loading State:
 - Il codice gestisce due stati distinti: `isLoadingIds` (per il bootstrap iniziale) e `isLoadingStories` (per la paginazione).
 - Beneficio: L'utente percepisce un avvio veloce, mentre i caricamenti successivi sono non intrusivi e non bloccano l'interfaccia.
- Error Handling:
 - Grazie all'operatore `catchError`, se una chiamata API fallisce o va in timeout, l'app non mostra schermate di errore allarmanti.
 - Le notizie "corrotte" o "morte" (dead o deleted) vengono filtrate silenziosamente prima del rendering, garantendo una lista pulita e leggibile.
- Visual Design: Stile "Retro Terminal" per un'immersione tematica coerente con il brand Hacker News.

Conclusioni, Link e Roadmap

📁 GitHub Repository (Codice & Setup): <https://github.com/sadsotti/hacker-news-hub>

🚀 Live App (Netlify Hosting): <https://hacker-news-hub.netlify.app>

Ipotetica Future Roadmap:

1. Angular Signals: Migrazione delle variabili di stato (`displayedStories`, `isLoading`) verso i Signals per una reattività fine-grained senza bisogno di chiamare manualmente `detectChanges()`.
2. Infinite Scroll: Sostituzione del pulsante "Load More" con l'API Observer per un flusso di lettura continuo.

